

HumanMaximizer AI Lead Generation

Architecture & Design Document

Razor Infotech Pvt Ltd — AI Architect / GenAI Engineer Assignment

Fine-Tuning Strategy

HumanMaximizer AI Lead Generation System

Razor Infotech Pvt Ltd — AI Architect / GenAI Engineer Assignment

Overview

This document explains the fine-tuning strategy for the HumanMaximizer AI Lead Generation system. The base model is **Mistral-7B-Instruct-v0.3** running via Ollama. The current production system uses **prompt engineering + RAG** as the primary approach. Fine-tuning is planned as a second-phase enhancement once sufficient training data is collected from live pipeline runs.

1. When Fine-Tuning is Better than Prompting

Fine-tuning is **not** the first-line solution for this system. The current RAG + prompting approach is preferred because:

Scenario	Prompting + RAG	Fine-Tuning
New product features added	Re-ingest website — no retraining	Full retraining cycle needed
Output format consistency	Jinja2 templates enforce structure	Bakes format into model weights
Domain vocabulary (CLRA, ESIC)	Inject via RAG chunks	Learned during training
Cold start (no training data)	Works immediately	Requires 300–500 examples minimum
Iteration speed	Change prompt in minutes	Hours of training per experiment

Trigger Points — When Fine-Tuning Becomes Justified

Fine-tuning for this system is triggered when **3 specific failure patterns** are observed consistently:

- **Malformed output structure** — LLM consistently produces broken `SUBJECT: /BODY:` splits requiring complex post-processing workarounds

- **Compliance hallucinations** — Qualification reasoning invents specific regulation names (e.g., "ESIC Form 9B") not present in any RAG chunk or training data
- **Generic outreach despite RAG** — Cold emails revert to placeholder language ("I noticed your company...") even when specific RAG chunks are injected with product facts

2. Fine-Tuning Approach — QLoRA with Unsloth

Why QLoRA over Full Fine-Tuning

Method	VRAM Required	Training Time	Quality
Full fine-tuning (FP16)	~80–120 GB	Days	Best
LoRA (FP16 base)	~24–32 GB	Hours	Very good
QLoRA (4-bit base)	~8–12 GB	Hours	Good — selected
Prompt engineering only	0 GB	Minutes	Acceptable

QLoRA freezes the base model weights in 4-bit quantization and trains small **low-rank adapter matrices (LoRA)** on top. This makes 7B model fine-tuning accessible on a single consumer GPU.

Unsloth accelerates QLoRA training **2–3× faster** than HuggingFace PEFT via:

- Custom Triton CUDA kernels for attention and MLP layers
- Gradient checkpointing optimizations
- Flash Attention 2 integration

LoRA Configuration

```
target_modules = [  
    "q_proj",    # query projection  
    "k_proj",    # key projection  
    "v_proj",    # value projection  
    "o_proj",    # output projection  
    "gate_proj", # feed-forward gate  
    "up_proj",   # feed-forward up  
    "down_proj", # feed-forward down  
]  
  
lora_r = 16          # rank – controls adapter expressiveness  
lora_alpha = 32      # scaling factor (typically 2 × rank)  
lora_dropout = 0.05  # regularization
```

Why rank=16: Rank controls how many parameters the adapter adds. Rank 16 adds ~13M trainable parameters on a 7B model (~0.2% of total). This is sufficient for task-specific adaptation without overfitting on small datasets.

3. Dataset Format

Fine-tuning dataset uses the **Alpaca JSONL format** — {instruction, input, output} triples.

Format Specification

instruction → task description (what to generate)
input → context (company data + RAG chunks)
output → ideal response (human-reviewed)

Example 1 — Cold Email Generation

```
{
  "instruction": "Write a personalised B2B cold email for an HRMS sale.",
  "input": "Company: Bharat Forge Ltd (Manufacturing, 10,000 employees, SAP HR + Oracle HRMS). Decision",
  "output": "SUBJECT: Modernise Multi-Plant HR at Bharat Forge\n\nBODY:\nDear Rajesh,\n\nBharat Forge's
```

Example 2 — Qualification Reasoning

```
{
  "instruction": "Write a 2-sentence lead qualification reasoning.",
  "input": "Company: Metropolis Healthcare Ltd (Healthcare, 4,000 employees, Excel + Google Sheets). Sco",
  "output": "Metropolis Healthcare's Excel-based HR across 200+ labs for 4,000 employees represents a h
```

Example 3 — Disqualification Reasoning

```
{
  "instruction": "Write a 2-sentence lead disqualification reasoning.",
  "input": "Company: Lusha (Unknown sector, unknown employees, no tech stack). Scores: company_size_fit=",
  "output": "Lusha's sector classification as Unknown combined with no verifiable employee count and zer
```

4. Training Dataset Construction

Data Collection Pipeline

- Step 1: Run live pipeline on 200-300 Indian B2B target companies
(SerpAPI: manufacturing, healthcare, BFSI, logistics, IT services)
↓
- Step 2: Human sales expert reviews each generated email + reasoning
Edits → marks as approved or rejected
↓
- Step 3: Approved outputs → training JSONL
Rejected outputs → adversarial JSONL (teach model what not to do)
↓
- Step 4: Add 50 synthetic adversarial examples:
 - Micro companies (< 10 employees) → model should score low
 - Irrelevant industries (agriculture, retail) → model should decline
 - Incomplete data → model should flag low confidence↓
- Step 5: Final dataset: 300-500 examples
Split: 80% train / 10% validation / 10% test

Target Dataset Size

Task	Examples Needed	Rationale
Cold email generation	150-200	High variation in company profiles

Task	Examples Needed	Rationale
Qualification reasoning	100–150	Structured output, fewer patterns
Disqualification reasoning	50–100	Edge cases and adversarial
Total	300–500	Quality >> quantity for instruction fine-tuning

5. Training Configuration

```

from unsloth import FastLanguageModel
from trl import SFTTrainer
from transformers import TrainingArguments

# Load base model in 4-bit (QLoRA)
model, tokenizer = FastLanguageModel.from_pretrained(
    model_name="mistralai/Mistral-7B-Instruct-v0.3",
    max_seq_length=4096,
    load_in_4bit=True,
    dtype=None, # auto-detect
)

# Attach LoRA adapters
model = FastLanguageModel.get_peft_model(
    model,
    r=16,
    lora_alpha=32,
    lora_dropout=0.05,
    target_modules=[
        "q_proj", "k_proj", "v_proj", "o_proj",
        "gate_proj", "up_proj", "down_proj"
    ],
    bias="none",
    use_gradient_checkpointing="unsloth",
    random_state=42,
)

# Training arguments
training_args = TrainingArguments(
    output_dir="./lora-humanmaximizer",
    num_train_epochs=3,
    per_device_train_batch_size=2,
    gradient_accumulation_steps=4, # effective batch = 8
    learning_rate=2e-4,
    warmup_ratio=0.1,
    lr_scheduler_type="cosine",
    fp16=True,
    logging_steps=10,
    save_steps=100,
    evaluation_strategy="steps",
    eval_steps=100,
    load_best_model_at_end=True,
)

trainer = SFTTrainer(
    model=model,
    tokenizer=tokenizer,
    train_dataset=train_dataset,
    eval_dataset=eval_dataset,
    dataset_text_field="text",

```

```
max_seq_length=4096,  
args=training_args,  
)  
  
trainer.train()
```

6. Deployment After Fine-Tuning

```
Train LoRA adapter  
↓  
Merge adapter into base weights  
model.save_pretrained_merged("merged-model", tokenizer, save_method="merged_4bit")  
↓  
Convert to GGUF Q4_K_M (Ollama format)  
llama.cpp: python convert.py merged-model --outtype q4_K_M  
↓  
Replace Ollama model  
ollama create humanmaximizer-ft -f Modelfile  
↓  
Update .env:  
OLLAMA_MODEL=humanmaximizer-ft  
↓  
Zero API changes – same endpoints, same prompts, better output quality
```

7. Evaluation Metrics

Metric	How Measured	Target
Email format compliance	SUBJECT: + BODY: present	> 98%
Email word count	150–220 words	> 95% in range
RAG grounding	Claims traceable to rag_context_used	> 90%
Hallucination rate	SelfCritiqueTool is_grounded=false	< 5%
Qualification accuracy	Score matches human reviewer label	> 85%
LinkedIn length	≤ 300 chars	> 99%

Summary

Phase	Approach	When
Now (Production)	Mistral-7B + Jinja2 prompts + RAG	Deployed
Phase 2	Collect 300–500 human-reviewed examples	After 1 month of live usage
Phase 3	QLoRA fine-tune with Unsloth on collected data	After dataset ready
Phase 4	Deploy fine-tuned GGUF via Ollama	Drop-in replacement

Fine-tuning improves **consistency and tone** — it does not replace RAG. Even after fine-tuning, the RAG pipeline continues to inject live product knowledge so outreach stays grounded in current HumanMaximizer features.